Kanishka Gupta
0801IT221074

ASSIGNMENT - 4

Q1: Write a program for Word Count

Implemented a MapReduce job in Java to count the frequency of words in a given text file.
It includes:

- Mapper class for tokenizing words
- Reducer class for counting word frequency
- Driver class to configure and execute the job

Step 1: Write the WordCount Java Code

Created the WordCount. java file containing the Mapper, Reducer, and Driver classes.

```
  GNU nano 7.2                                                    WordCount.java *
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable> {
        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context) throws IOException, InterruptedException {
            StringTokenizer itr = new StringTokenizer(value.toString());
            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken());
                context.write(word, one);
            }
        }
    }

    public static class IntSumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
        private IntWritable result = new IntWritable();

        public void reduce(Text key, Iterable<IntWritable> values, Context context)
                throws IOException, InterruptedException {
            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get();
            }
            result.set(sum);
            context.write(key, result);
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "word count");

        job.setJarByClass(WordCount.class);
        job.setMapperClass(TokenizerMapper.class);
        job.setCombinerClass(IntSumReducer.class);
        job.setReducerClass(IntSumReducer.class);

^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark  M-]
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy      ^Q W
```

Step 2: Compile and Package into a JAR

Compiled the Java code using the Hadoop classpath and packaged the output classes into wordcount.jar .

Kanishka Gupta
0801IT221074

```
kanishka@master:~$ nano WordCount.java
kanishka@master:~$ javac -classpath $(hadoop classpath) -d . WordCount.java
kanishka@master:~$ jar -cvf wordcount.jar *
added manifest
adding: WordCount$IntSumReducer.class(in = 1739) (out= 742)(deflated 57%)
adding: WordCount$TokenizerMapper.class(in = 1736) (out= 752)(deflated 56%)
adding: WordCount.class(in = 1491) (out= 814)(deflated 45%)
adding: WordCount.java(in = 2186) (out= 705)(deflated 67%)
adding: assi4/(in = 0) (out= 0)(stored 0%)
adding: downloaded.txt(in = 24) (out= 26)(deflated -8%)
adding: file.txt(in = 24) (out= 26)(deflated -8%)
adding: hadoop-3.3.6.tar.gz(in = 730107476) (out= 728992709)(deflated 0%)
```

Step 3: Run the MapReduce Job and View Output

Input data was uploaded to HDFS, and the MapReduce job was executed using the Hadoop command. The output was stored in HDFS and displayed using the -cat command.

```
kanishka@master:~$ echo "Hadoop is an open source framework. Hadoop is used for big data processing. MapReduce is a progra
mming model in Hadoop. HDFS is the storage system of Hadoop. Big data requires distributed systems like Hadoop. Hadoop pro
vides scalability and fault tolerance" >input.txt
kanishka@master:~$ hdfs dfs -mkdir /assi4_word_count_input
kanishka@master:~$ hdfs dfs -put input.txt /assi4_word_count_input
kanishka@master:~$
```

```
kanishka@master:~$ hadoop jar wordcount.jar WordCount /assi4_word_count_input /wc_output
2026-04-22 11:03:05,560 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-04-22 11:03:05,935 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute y
our application with ToolRunner to remedy this.
2026-04-22 11:03:06,005 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/kanishka/.staging/job_1776855503
981_0002
2026-04-22 11:03:08,105 INFO input.FileInputFormat: Total input files to process : 1
2026-04-22 11:03:08,582 INFO mapreduce.JobSubmitter: number of splits:1
2026-04-22 11:03:08,705 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1776855503981_0002
2026-04-22 11:03:08,705 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-04-22 11:03:08,837 INFO conf.Configuration: resource-types.xml not found
2026-04-22 11:03:08,837 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-04-22 11:03:09,282 INFO impl.YarnClientImpl: Submitted application application_1776855503981_0002
2026-04-22 11:03:09,336 INFO mapreduce.Job: The url to track the job: http://localhost:8088/proxy/application_1776855503981_0002/
2026-04-22 11:03:09,336 INFO mapreduce.Job: Running job: job_1776855503981_0002
2026-04-22 11:03:18,428 INFO mapreduce.Job: Job job_1776855503981_0002 running in uber mode : false
2026-04-22 11:03:18,429 INFO mapreduce.Job:  map 0% reduce 0%
2026-04-22 11:03:23,217 INFO mapreduce.Job:  map 100% reduce 0%
2026-04-22 11:03:27,261 INFO mapreduce.Job:  map 100% reduce 100%
2026-04-22 11:03:27,273 INFO mapreduce.Job: Job job_1776855503981_0002 completed successfully
2026-04-22 11:03:27,357 INFO mapreduce.Job: Counters: 54
```

```
kanishka@master:~$ hdfs dfs -cat /wc_output/part-r-00000
Big     1
HDFS    1
Hadoop  3
Hadoop. 3
MapReduce       1
a       1
an      1
and     1
big     1
data    2
distributed     1
fault   1
for     1
framework.      1
in      1
is      4
like    1
model   1
of      1
open    1
processing.     1
programming     1
provides        1
requires        1
scalability     1
source  1
storage 1
system  1
systems 1
the     1
tolerance       1
used    1
kanishka@master:~$
```

Kanishka Gupta
0801IT221074

Q2: Write a program to find an anagram of a word

Implemented a MapReduce job that groups words containing the exact same characters.

Step 1: Write the Anagram Java Code

Created the Anagram.java file. The Mapper sorts characters alphabetically to form a key, and the Reducer groups the original words by that key.

```
  GNU nano 7.2                                                    Anagram.java *
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class Anagram {
    public static class AnagramMapper extends Mapper<Object, Text, Text, Text> {
        private Text sortedKey = new Text();
        private Text word = new Text();

        public void map(Object key, Text value, Context context)
                throws IOException, InterruptedException {

            String line = value.toString();
            String[] words = line.split("\\s+");

            for (String w : words) {
                char[] chars = w.toCharArray();
                Arrays.sort(chars);

                sortedKey.set(new String(chars));
                word.set(w);
                context.write(sortedKey, word);
            }}
    }

    public static class AnagramReducer extends Reducer<Text, Text, Text, Text> {

        public void reduce(Text key, Iterable<Text> values, Context context) throws IOException, InterruptedException {
            StringBuilder result = new StringBuilder();
            for (Text val : values) {
                result.append(val.toString()).append(" ");
            }
            context.write(key, new Text(result.toString()));
        }
    }

    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Anagram Finder");

        job.setJarByClass(Anagram.class);

        job.setMapperClass(AnagramMapper.class);
        job.setReducerClass(AnagramReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark  M-] To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy      ^Q Where Was
```

Step 2: Run the Job and View Output

Compiled the code, uploaded an input file of scrambled words, executed the MapReduce job, and displayed the grouped anagrams.

Kanishka Gupta
0801IT221074

```
kanishka@master:~$ javac -classpath `hadoop classpath` -d . Anagram.java
kanishka@master:~$ jar -cvf anagram.jar *
added manifest
adding: Anagram$AnagramMapper.class(in = 1815) (out= 832)(deflated 54%)
adding: Anagram$AnagramReducer.class(in = 1736) (out= 741)(deflated 57%)
adding: Anagram.class(in = 1408) (out= 769)(deflated 45%)
adding: Anagram.java(in = 2170) (out= 718)(deflated 66%)
adding: WordCount$IntSumReducer.class(in = 1739) (out= 742)(deflated 57%)
adding: WordCount$TokenizerMapper.class(in = 1736) (out= 752)(deflated 56%)
adding: WordCount.class(in = 1491) (out= 814)(deflated 45%)
adding: WordCount.java(in = 2186) (out= 705)(deflated 67%)
adding: assi4/(in = 0) (out= 0)(stored 0%)
adding: downloaded.txt(in = 24) (out= 26)(deflated -8%)
adding: file.txt(in = 24) (out= 26)(deflated -8%)
adding: hadoop-3.3.6.tar.gz(in = 730107476) (out= 728992709)(deflated 0%)
adding: input.txt(in = 257) (out= 168)(deflated 34%)
adding: wordcount.jar(in = 728997102) (out= 729218362)(deflated 0%)
kanishka@master:~$ nano input.txt
kanishka@master:~$ hdfs dfs -mkdir /anagram_input
kanishka@master:~$ hdfs dfs -put input.txt /anagram_input
kanishka@master:~$ hadoop jar anagram.jar Anagram /anagram_input /anagram_output
2026-04-22 11:15:21,704 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-04-22 11:15:22,349 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute y
our application with ToolRunner to remedy this.
2026-04-22 11:15:22,374 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/kanishka/.staging/job_1776855503
981_0003
2026-04-22 11:15:25,903 INFO input.FileInputFormat: Total input files to process : 1
2026-04-22 11:15:26,776 INFO mapreduce.JobSubmitter: number of splits:1
2026-04-22 11:15:26,918 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1776855503981_0003
2026-04-22 11:15:26,918 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-04-22 11:15:27,060 INFO conf.Configuration: resource-types.xml not found
2026-04-22 11:15:27,060 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-04-22 11:15:27,110 INFO impl.YarnClientImpl: Submitted application application_1776855503981_0003
2026-04-22 11:15:27,145 INFO mapreduce.Job: The url to track the job: http://localhost:8088/proxy/application_1776855503981_0003/
2026-04-22 11:15:27,146 INFO mapreduce.Job: Running job: job_1776855503981_0003
2026-04-22 11:15:39,009 INFO mapreduce.Job: Job job_1776855503981_0003 running in uber mode : false
2026-04-22 11:15:39,013 INFO mapreduce.Job:  map 0% reduce 0%
2026-04-22 11:15:42,093 INFO mapreduce.Job:  map 100% reduce 0%
2026-04-22 11:15:46,124 INFO mapreduce.Job:  map 100% reduce 100%
2026-04-22 11:15:47,150 INFO mapreduce.Job: Job job_1776855503981_0003 completed successfully
2026-04-22 11:15:47,233 INFO mapreduce.Job: Counters: 54
```

```
                Bytes Written=62
kanishka@master:~$ hdfs dfs -cat /anagram_output/part-r-00000

act     act tac cat
dgo     god dog
eilnst  enlist silent listen
kanishka@master:~$
```